

解构SSH密钥交换

深入Rust客户端实现的技术解析

我们的目标：在不安全的网络上构建一座加密堡垒

在深入代码之前，我们必须明确目标。SSH密钥交换是一个精心设计的流程，旨在在一个不可信的网络（如互联网）上建立一个完全安全的通信通道。成功完成后，我们将获得四个关键成果：



- **算法协商 (Algorithm Negotiation)**：客户端和服务端就加密、认证和密钥交换方法达成一致。



- **共享密钥 (Shared Secret)**：使用Diffie-Hellman算法安全地生成一个双方都知道但从未在网络上传输的秘密值 **(K)**。



- **会话密钥 (Session Keys)**：从共享密钥派生出一整套用于加密和验证每个数据包的密钥。



- **会话ID (Session ID)**：创建一个唯一的标识符 **(H)**，作为本次连接的加密指纹。

旅程概览：密钥交换协议全景图

我们将把这个过程分为三个主要阶段。这张图是我们的路线图，我们将在接下来的幻灯片中逐步深入每个步骤。



第一阶段：协商的握手（KEXINIT）

寻找共同语言

交换的第一步是客户端和服务端交换 SSH_MSG_KEXINIT 消息。这就像双方各自亮出自己支持的算法“菜单”。此消息包含：

- 一个16字节的随机 cookie（防止重放攻击）。
- 支持的密钥交换、主机密钥、加密和MAC算法列表。
- 其他协议标志。

目的是从这些列表中找出双方都支持的第一套通用算法。

Rust 实现

```
// 1. 创建客户端的 KEXINIT 消息
let client_kex = KexInit::new();
let client_kex_bytes = client_kex.to_bytes();

// 2. 将其封装并发送到服务器
write_packet(&mut stream, &client_kex_bytes)
    .context("Failed to send client KEXINIT")?;

// 3. 读取并解析服务器的 KEXINIT 消息
let server_kex_payload = read_packet(&mut stream)?;
let server_kex =
    KexInit::from_bytes(&server_kex_payload)?;

negotiate_algorithms(&client_kex, &server_kex)
```


达成的共识：我们选择的加密语言

协商函数 ``negotiate_algorithms`` 会比较客户端和服务器的列表，并为每个类别选择第一个共同支持的算法。顺序至关重要。

Negotiated algorithms:

KEX: diffie-hellman-group14-sha256

Host key: ssh-rsa

Encryption (C->S): aes256-ctr

Encryption (S->C): aes256-ctr

MAC (C->S): hmac-sha2-256

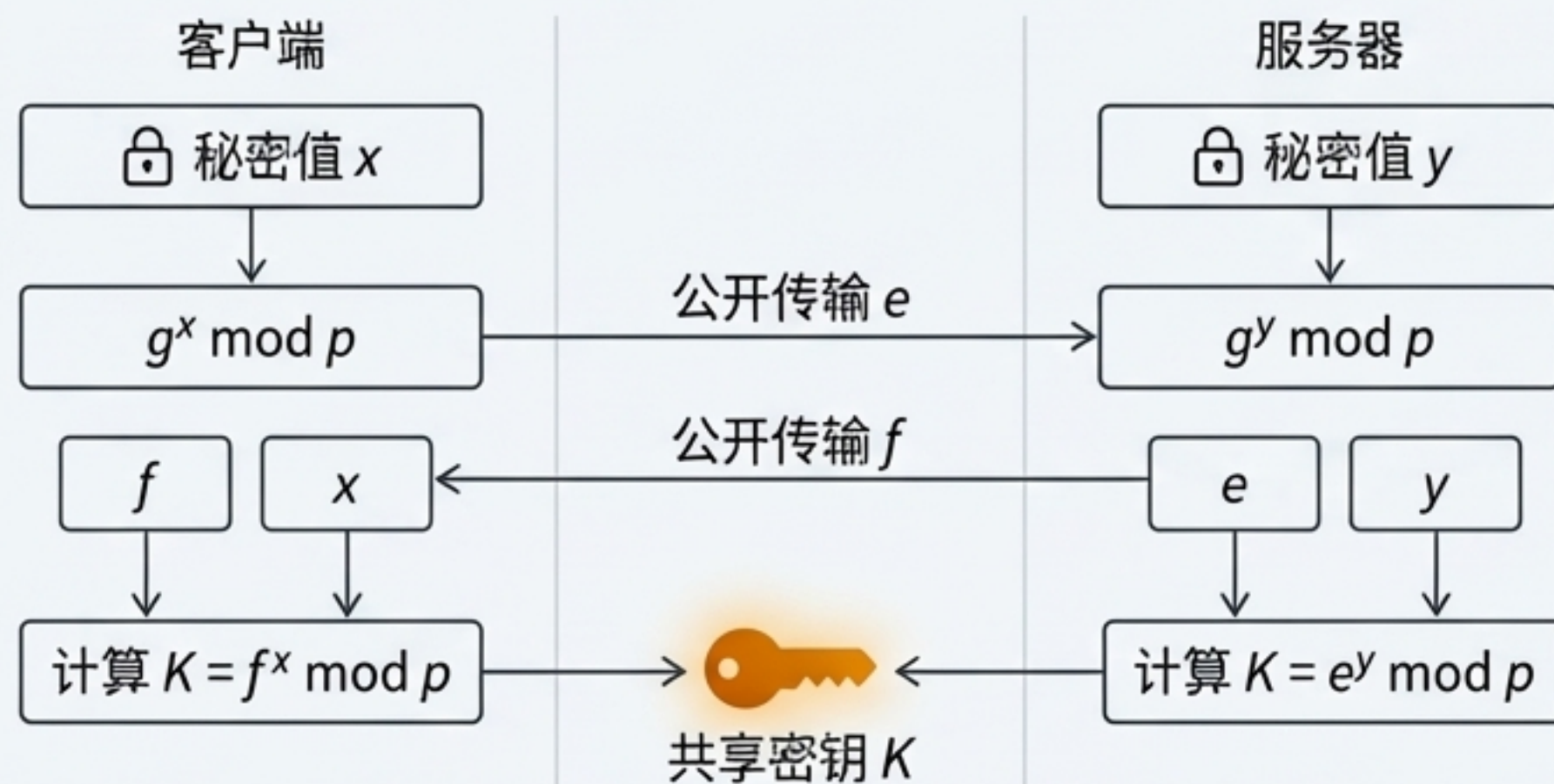
MAC (S->C): hmac-sha2-256

这些就是接下来将用于构建安全通道的具体算法。每一步都有了明确的“配方”。

第二阶段：Diffie-Hellman 的舞台

核心挑战：如何在众目睽睽之下，创造出一个只有你和服务器知道的秘密？

Diffie-Hellman 密钥交换允许双方在一个不安全的信道上共同建立一个共享密钥，而无需直接传输该密钥本身。



$$(g^y)^x = (g^x)^y = g^{xy} \bmod p$$

双方最终得到完全相同的共享密钥 K ，但 K 本身从未在网络上传输过。

客户端的第一步：发送公钥 e

客户端首先在本地执行以下操作：

1. 加载 Diffie-Hellman Group14 的大素数 p 和生成器 g 。
2. 生成一个安全的随机私钥 x 。
3. 计算公钥 $e = g^x \bmod p$ 。

Rust 实现快照

```
// 客户端生成DH密钥对
let client_dh = DiffieHellman::new();
let client_public_key_bytes = client_dh.public_key_bytes();

// 构建 SSH_MSG_KEXDH_INIT 消息 (ID: 30)
let mut dh_message = Vec::new();
dh_message.push(30);
dh_message.extend_from_slice(&client_public_key_bytes);

// 发送消息
write_packet(stream, &dh_message)?;
```

消息负载结构

消息ID	公钥长度	公钥 'e' (字节流)
30	00 00 01 00	A3 B1 C4 E2 F5 A9 D8 B7 C6...

发送到网络

服务器的关键回应：解析 `KEXDH\_REPLY`

服务器的回应 `SSH\_MSG\_KEXDH\_REPLY` (ID: 31) 是整个流程中最关键的数据包之一。客户端必须精确地解析这个字节流，以提取三个至关重要的部分。

| 31 | K_S_len | K_S | f_len | f | sig_len | sig |

1. 服务器主机密钥 (K_S)

服务器的长期公钥（例如，`/etc/ssh/ssh\_host\_rsa\_key.pub` 的内容）。它将用于验证服务器的身份。

```
let k_s = &server_dh_payload  
[offset..offset + k_s_length];
```

2. 服务器DH公钥 (f)

服务器为此会话生成的临时 DH 公钥。客户端将用它来计算共享密钥。

```
let server_public_key =  
DiffieHellman::parse_public_key(  
...)
```

3. 签名 (Signature)

服务器使用其主机私钥对交换过程的关键部分近的数字签名。**这是验证服务器身份、防止中间人攻击的核心机制。**

```
let _signature =  
&server_dh_payload[offset..offset  
+ _signature_length];
```

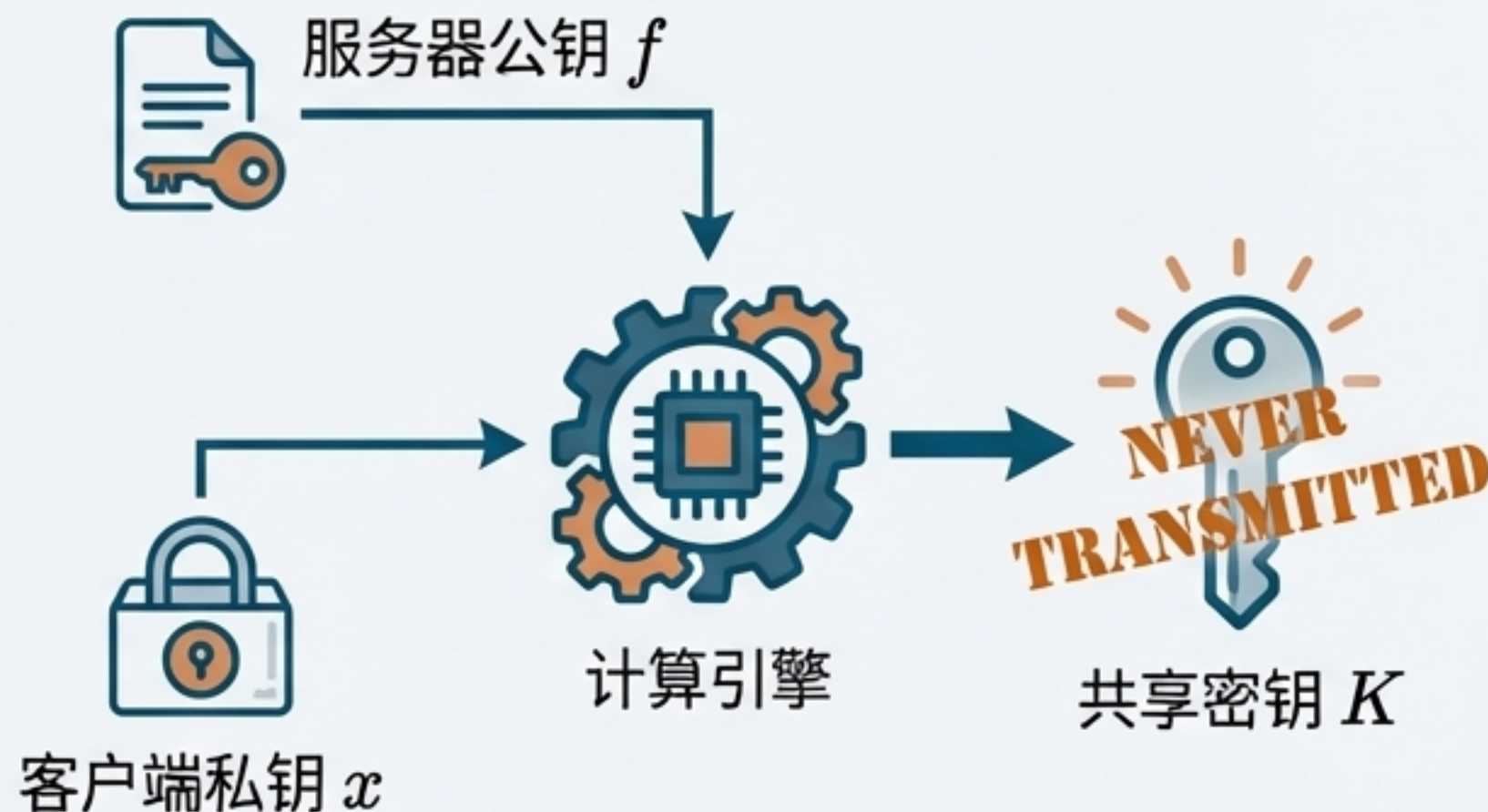
在我们的示例中，我们解析了签名但跳过了验证。在生产环境中，**必须验证**此签名。

秘密的诞生：计算共享密钥 K

在成功解析服务器的公钥 f 后，客户端现在拥有了计算共享密钥 K 所需的所有信息。这是 Diffie-Hellman 交换的巅峰时刻。

$$K = f^x \bmod p$$

客户端使用服务器的公钥 f 和自己的私钥 x 进行计算。这个计算完全在客户端本地发生。



// 这就是 Diffie-Hellman 的核心!

```
let shared_secret = client_dh.compute_shared_secret(&server_public_key);  
println!("Computed shared secret");
```


信任的基石：交换哈希 H

除了共享密钥 K 之外，我们还需要计算第二个关键值：交换哈希 H 。 H 是对整个密钥交换过程所有关键参数的加密摘要。它确保了交换的完整性，并将所有部分绑定在一起。



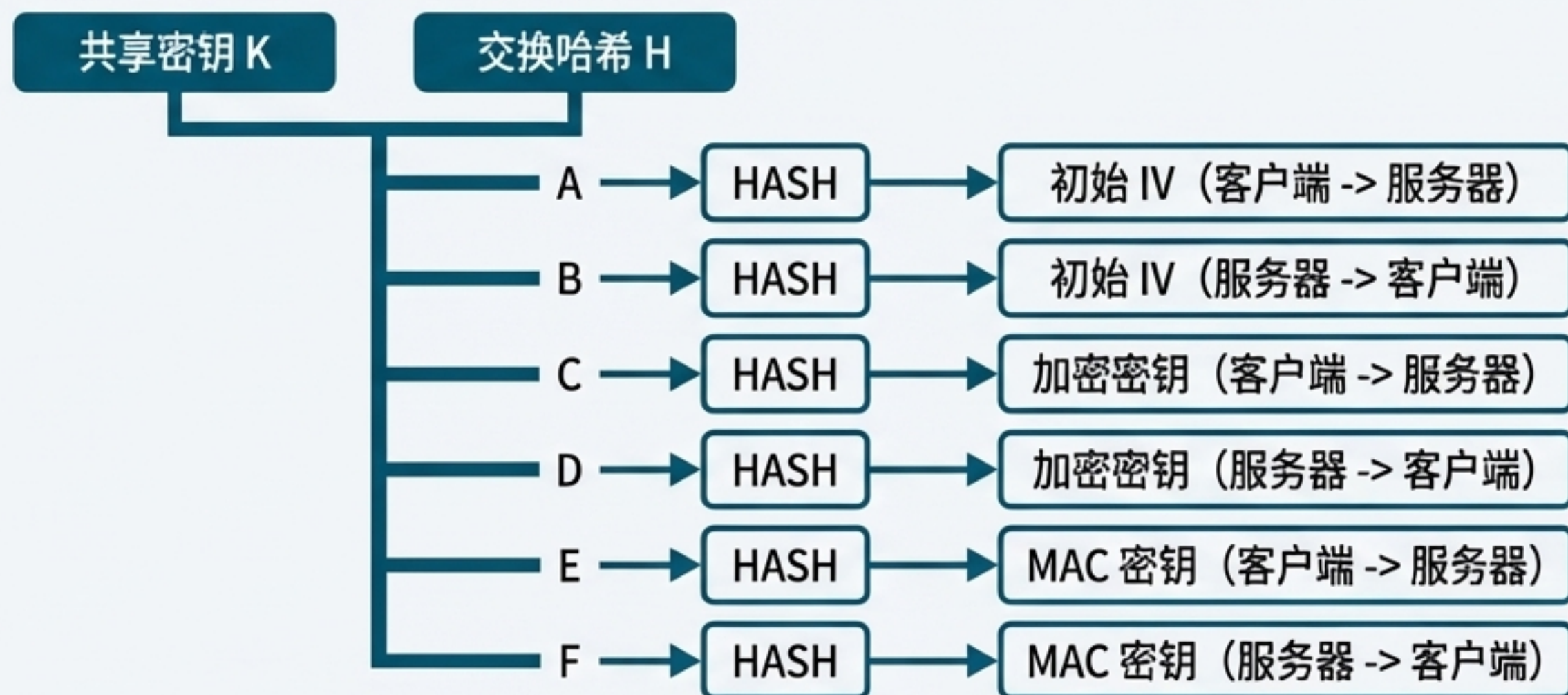
Key Roles of H :

- 它本身就是本次连接的**会话ID (Session ID)**。
- 它是生成所有会话加密密钥和MAC密钥的**关键输入**。

第三阶段：铸造会话密钥

有了共享密钥 K 和交换哈希 H 这两个基础秘密，我们现在可以派生出加密和认证数据所需的所有会话密钥。这确保了每个方向、每种用途都有一个独立的密钥。

```
key = HASH(K || H || letter || session_id) (其中 session_id 就是 H)
```



```
let session_keys = derive_session_keys(&shared_secret, &h, &session_id);
```


开启加密：交换 NEWKEYS

密钥交换的最后一步是双方交换 SSH_MSG_NEWKEYS 消息（ID：21）。

这条消息本身不包含数据，它是一个信号。

- **发送 NEWKEYS**** 意味着：“我已经生成了所有密钥，从现在开始我发送的所有数据都将使用新密钥加密。”
- **接收 NEWKEYS**** 意味着：“我已收到你的信号，从现在开始我接收的所有数据都将使用新密钥解密。”



交换完成后，一个双向的、完全加密和认证的通道就建立起来了。旅程完成！

全景回顾：协议流程与代码映射

我们已经走完了整个密钥交换的旅程。下表将我们的叙事阶段与 Rust 实现中的具体代码行对应起来，方便您回顾源代码。

阶段	代码行	目的
第一阶段：协商	17-24	发送客户端 KEXINIT
协商 (Lines 17-40)	26-31	接收并解析服务器 KEXINIT
	33-40	协商通用算法
第二阶段：加密核心		
加密核心 (Lines 42-143)	68-77	客户端生成并发送 DH 公钥 e
	79-90	接收服务器的 KEXDH_REPLY 响应
	92-125	解析服务器的主机密钥 K_S、公钥 f 和签名
	129-131	计算共享密钥 K
	133-143	计算交换哈希 H
第三阶段：定局与激活		
定局与激活 (Lines 145-169)	145-150	从 K 和 H 派生会话密钥
	152-165	交换 NEWKEYS 消息，激活加密
	169	成功返回协商结果和会话密钥

任务完成：核心安全收益与技术亮点

实现的安全属性

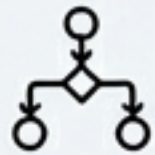
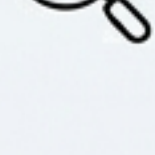
- 🕒 **前向保密** (Forward Secrecy)：每个会话都有独立的、临时生成的密钥。即使服务器的长期私钥泄露，过去的会话数据仍然是安全的。
- 🔒 **服务器认证** (Server Authentication)：通过验证服务器主机密钥的签名（在生产代码中），客户端可以确信它正在与正确的服务器通信，而不是中间人。
- 📄 **完整性** (Integrity)：交换哈希 `H` 将所有协商参数绑定在一起，任何一方的参数被篡改都会导致哈希不匹配，从而使交换失败。

关键 Rust 概念

- ❓ **Result 和 ? 运算符**：实现健壮、清晰的错误处理。
- & **所有权与借用** (`&`)：安全地管理内存和网络资源。
- 01010101 **模式匹配与验证**：精确解析二进制协议，确保数据格式正确。
- 🗨️ **anyhow::Context**：为错误添加上下文，极大地方便了调试。

前路漫漫：从示例到生产环境的考量

虽然我们的代码成功演示了密钥交换协议，但一个生产级的 SSH 客户端还需要考虑更多。以下是构建一个真正健壮系统所必需的关键步骤：

-  **签名验证 (Signature Validation)：** **这是最重要的安全步骤。** 必须使用服务器的主机公钥 `K_S` 验证其签名，以防止中间人攻击。
-  **完整的算法支持 (Algorithm Support)：** 支持多种密钥交换、加密和MAC算法，并根据协商结果动态选择。
-  **稳健的错误处理 (Error Recovery)：** 实现更复杂的错误处理和重试逻辑，而不仅仅是 `bail!`。
-  **会话重协商 (Re-keying)：** 支持在长时间运行的会话中定期重新执行密钥交换。
-  **专业的日志记录 (Logging)：** 使用结构化日志库（如 `tracing` 或 `log`）代替 `println!`。
-  **安全审计 (Security Audits)：** 任何加密相关的代码都应由外部专家进行严格审查。